Computer Architecture: Introduction

Tatpong Katanyukul

CoE, KKU

4 กันยายน พ.ศ. 2558

The World Fastest Computer



- As of Nov 2014, TIANHE-2 (MILKYWAY-2): 33.86 petaflop/s (Linpack benchmark)
- 16,000 nodes: each = 2 Intel Ivy Bridge Xeon + 3 Xeon Phi
  Xeon Phi = coprocessor
- Total memory 1,375 TiB ($= 1.511828 \times 10^{15} \approx 1,511$ TB)

src: http://www.top500.org/featured/top-systems/

Performance

Airplane	Passenger capacity	Cruising range (miles)	Cruising speed (m.p.h.)	Passenger throughput (passengers × m.p.h.)
Boeing 777	375	4630	610	228,750
Boeing 747	470	4150	610	286,700
BAC/Sud Concorde	132	4000	1350	178,200
Douglas DC-8-50	146	8720	544	79,424

FIGURE 1.14 The capacity, range, and speed for a number of commercial airplanes. The last column shows the rate at which the airplane transports passengers, which is the capacity times the cruising speed (ignoring range and takeoff and landing times).

Performance

- desktop computer: time to get job done

  response time (a.k.a. execution time): total time to complete a task
- datacenter: number of jobs done during a day

  throughput (a.k.a. bandwidth): number of tasks completed per unit time
- FLOPs = Floating-Point Operations per second

Response Time as Performance Measurement

$$\text{Performance}_x = \frac{1}{\text{Execution time}_x}.$$

เปรียบเทียบ performance ของ เครื่อง X กับ เครื่อง Y, ถ้า

$$\begin{aligned} \text{Execution time}_X &< \text{Execution time}_Y \\ \frac{1}{\text{Execution time}_X} &> \frac{1}{\text{Execution time}_Y} \\ \text{Performance}_X &> \text{Performance}_Y \end{aligned}$$

เครื่อง X เร็วกว่า เครื่อง Y.

Response Time as Performance Measurement

$$\frac{\text{Performance}_X}{\text{Performance}_Y} = n$$

หรือ

$$\frac{\text{Execution time}_Y}{\text{Execution time}_X} = n$$

$\Rightarrow$ เครื่อง X เร็วเป็น n เท่า ของเครื่อง Y.

ตัวอย่าง เครื่อง A รันโปรแกรม ก เสร็จในเวลา 10 วินาที และ เครื่อง B รันเสร็จในเวลา 15 วินาที, เครื่อง A เร็วเป็น $\frac{15}{10} = 1.5$ เท่าของเครื่อง B.

Jargons

- CPU execution time (CPU time): the actual time the CPU spends computing for a specific task.
- user CPU time: the CPU time spent in a program itself.
- system CPU time: the CPU time spent in the OS performing tasks on behalf of the program.
- clock cycle (a.k.a. clock period): the time for one clock period of the processor clock

  E.g., a processor with 3.6GHz (clock rate $= 3.6$ GHz) $\Rightarrow$ clock period $= \frac{1}{3.6G} \approx 278$ ps.

CPU Time

$$\begin{aligned} \text{CPU time} &= \text{Number of CPU cycles} \times \text{Clock period} \\ &= \frac{\text{Number of CPU cycles}}{\text{Clock rate}} \end{aligned}$$

Improving Performance

- Computer A runs Program 1 in 10 seconds
  Computer A: 2 GHz
- Plan to build Computer B to finish Program in 6 seconds
  Increasing Computer B clock rate results in increasing clock cycle:
  Compute B needs 1.2 times as many as clock cycle as Computer A
- To meet a plan, what should clock rate of Computer B should be?

Improving Performance

- Computer A runs Program 1 in 10 seconds
  Computer A: 2 GHz
- Plan to build Computer B to finish Program in 6 seconds
  Increasing Computer B clock rate results in increasing clock cycle:
  Compute B needs 1.2 times as many as clock cycle as Computer A
- To meet a plan, what should clock rate of Computer B should be?

$$\begin{aligned} \text{CPU time}_A &= \frac{\text{Clock cycles}_A}{\text{Clock rate}_A} \\ 10 &= \frac{N_A}{2 \times 10^9} \end{aligned}$$

Improving Performance

$$
\begin{aligned}
\text{CPU time}_A &= \frac{\text{Clock cycles}_A}{\text{Clock rate}_A} \\
10 &= \frac{N_A}{2 \times 10^9} \\
N_A &= 20 \times 10^9 \text{cycles} \\
\text{CPU time}_B &= \frac{\text{Clock cycles}_B}{\text{Clock rate}_B} \\
6 &= \frac{1.2 \cancel{N_A}^{20G}}{\text{Clock rate}_B} \\
\text{Clock rate}_B &= 4GHz
\end{aligned}
$$

CPI

- clock cycles per instruction (CPI):
  average number of clock cycles per instruction for a program or program fragment
- instruction count:
  the number of instructions executed by the program

$$\text{CPU clock cycles} = \text{Number of instructions} \times \text{CPI}$$

$$\text{CPU time} = \frac{\text{Instruction count} \times \text{CPI}}{\text{Clock rate}}$$

CPU Time

$$\text{Time} = \frac{\text{Instruction count} \times \text{CPI}}{\text{Clock rate}}$$

$$\text{Time} = \text{Seconds/Program} = \frac{\text{Instructions}}{\text{Program}} \times \frac{\text{Clock cycles}}{\text{Instructions}} \times \frac{\text{Seconds}}{\text{Clock cycles}}$$

Comparing Code Segments

- A compiler designer is choosing between 2 code sequences.

Code sequences	Instruction counts		
	Class A	Class B	Class C
1	2	1	2
2	4	1	1

	CPI for each class		
	Class A	Class B	Class C
CPI	1	2	3

Comparing Code Segments

	CPI for each class		
	Class A	Class B	Class C
CPI	1	2	3

Code sequences	Instruction counts		
	Class A	Class B	Class C
1	2	1	2
2	4	1	1

- Number of instructions

  Seq 1: $2 + 1 + 2 = 5$ instructions

  Seq 2: $4 + 1 + 1 = 6$ instructions

Comparing Code Segments

	CPI for each class		
	Class A	Class B	Class C
CPI	1	2	3

Code sequences	Instruction counts		
	Class A	Class B	Class C
1	2	1	2
2	4	1	1

- Number of instructions

  Seq 1: $2 + 1 + 2 = 5$ instructions

  Seq 2: $4 + 1 + 1 = 6$ instructions

- Number of clock cycles

  Seq 1: $2 \times 1 + 1 \times 2 + 2 \times 3 = 10$ cycles

  Seq 2: $4 \times 1 + 1 \times 2 + 1 \times 3 = 9$ cycles

Comparing Code Segments

	CPI for each class		
	Class A	Class B	Class C
CPI	1	2	3

Code sequences	Instruction counts		
	Class A	Class B	Class C
1	2	1	2
2	4	1	1

- Number of instructions
  Seq 1: $2 + 1 + 2 = 5$ instructions
  Seq 2: $4 + 1 + 1 = 6$ instructions
- Number of clock cycles
  Seq 1: $2 \times 1 + 1 \times 2 + 2 \times 3 = 10$ cycles
  Seq 2: $4 \times 1 + 1 \times 2 + 1 \times 3 = 9$ cycles

- clock cycle time has traditionally been fixed.
- today's processors can vary their clock rates
- Core i7 will temporary increase clock rate by about 10% until it gets too warm.
- Intel calls this "Turbo" mode.

Program Performance

Hardware or software component	Affects what?	How?
Algorithm	Instruction count, possibly CPI	The algorithm determines the number of source program instructions executed and hence the number of processor instructions executed. The algorithm may also affect the CPI, by favoring slower or faster instructions. For example, if the algorithm uses more divides, it will tend to have a higher CPI.
Programming language	Instruction count, CPI	The programming language certainly affects the instruction count, since statements in the language are translated to processor instructions, which determine instruction count. The language may also affect the CPI because of its features; for example, a language with heavy support for data abstraction (e.g., Java) will require indirect calls, which will use higher CPI instructions.
Compiler	Instruction count, CPI	The efficiency of the compiler affects both the instruction count and average cycles per instruction, since the compiler determines the translation of the source language instructions into computer instructions. The compiler's role can be very complex and affect the CPI in complex ways.
Instruction set architecture	Instruction count, clock rate, CPI	The instruction set architecture affects all three aspects of CPU performance, since it affects the instructions needed for a function, the cost in cycles of each instruction, and the overall clock rate of the processor.

The Power Wall

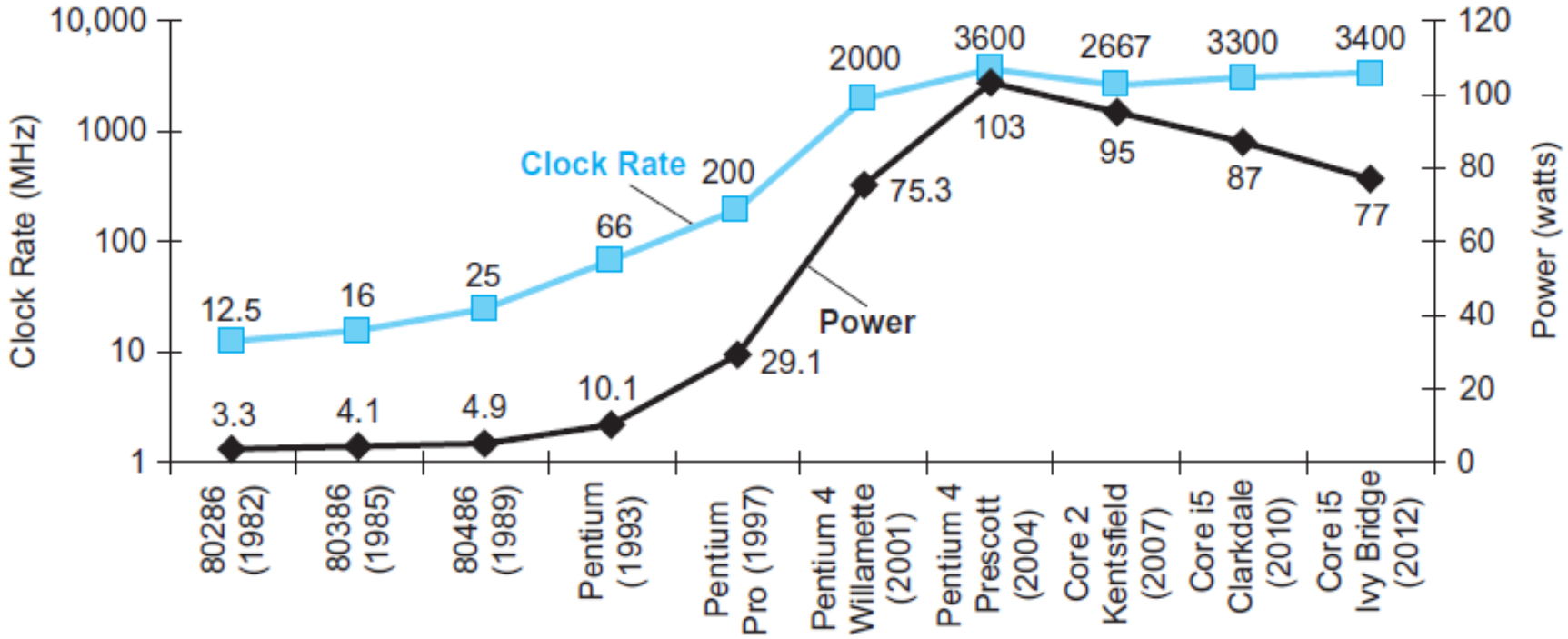


FIGURE 1.16 Clock rate and Power for Intel x86 microprocessors over eight generations and 25 years. The Pentium 4 made a dramatic jump in clock rate and power but less so in performance. The Prescott thermal problems led to the abandonment of the Pentium 4 line. The Core 2 line reverts to a simpler pipeline with lower clock rates and multiple processors per chip. The Core i5 pipelines follow in its footsteps.

Switching from Uniprocessors to Multiprocessors



- $\sim$ 2006, microprocessors with multiple processors per chip
- benefit is more on throughout than on response time
- processors in a chip are called “cores”
- the chip is called a “multicore microprocessor”

Benchmarking Core i7

Benchmark:

- a program selected for use in comparing computer performance.
- benchmark $\sim$ workload hope to predict the performance of actual workload

SPEC (System Performance Evaluation Cooperative):

- supported by computer vendors
- creates standard sets of benchmarks for computers.
- E.g., SPEC89, ..., SPEC CPU2006.

Benchmarking Core i7

SPEC CPU2006

- 12 integer benchmarks (CINT2006)
  vary from a part of C compiler to chess program to quantum computer simulation
- 17 floating-point benchmarks (CFP2006)
  finite element models, molecular dynamics, sparse linear algebra for fluid dynamics

Benchmarking Core i7

Description	Name	Instruction Count x 10^9	CPI	Clock cycle time (seconds x 10^{-9})	Execution Time (seconds)	Reference Time (seconds)	SPECratio
Interpreted string processing	perl	2252	0.60	0.376	508	9770	19.2
Block-sorting compression	bzip2	2390	0.70	0.376	629	9650	15.4
GNU C compiler	gcc	794	1.20	0.376	358	8050	22.5
Combinatorial optimization	mcf	221	2.66	0.376	221	9120	41.2
Go game (AI)	go	1274	1.10	0.376	527	10490	19.9
Search gene sequence	hmmer	2616	0.60	0.376	590	9330	15.8
Chess game (AI)	sjeng	1948	0.80	0.376	586	12100	20.7
Quantum computer simulation	libquantum	659	0.44	0.376	109	20720	190.0
Video compression	h264avc	3793	0.50	0.376	713	22130	31.0
Discrete event simulation library	omnetpp	367	2.10	0.376	290	6250	21.5
Games/path finding	astar	1250	1.00	0.376	470	7020	14.9
XML parsing	xalancbmk	1045	0.70	0.376	275	6900	25.1
Geometric mean	–	–	–	–	–	–	25.7

FIGURE 1.18 SPECINTC2006 benchmarks running on a 2.66 GHz Intel Core i7 920. As the equation on page 35 explains, execution time is the product of the three factors in this table: instruction count in billions, clocks per instruction (CPI), and clock cycle time in nanoseconds. SPECratio is simply the reference time, which is supplied by SPEC, divided by the measured execution time. The single number quoted as SPECINTC2006 is the geometric mean of the SPECratios.

Benchmarking Core i7

- SPECratio = Execution time of a reference computer / Execution time of the measured computer

Geometric mean,

$$\bar{x} = \sqrt[n]{\prod_{i=1}^{n} x_i}$$

```
> x = c(19.2, 15.4, 22.5, 41.2, 19.9, 15.8, 20.7, 190, 31, 21.5, ↩
    14.9, 25.1)
> prod(x)            ##  ∏_{i=1}^{12} x_i
[1] 8.448818e+16     ##  = 19.2 × 15.4 × ... × 25.1
> length(x)
[1] 12
> prod(x)^{1/12}     ##  ∛(∏_{i=1}^{12} x_i)
[1] 25.7375
```

Geometric mean v.s. Arithmetic mean

เครื่องไหนเร็วกว่า?

	A	B	C	D	E	F	G	H	I
1		Exe. Time			Perf Ratio	Ref = A		Perf Ratio	Ref = B
2		Mach A	Mach B		Mach A	Mach B		Mach A	Mach B
3	Prog 1	10	10		1	1		1	1
4	Prog 2	1200	300		1	4		0.25	1
5	Prog 3	733	2000		1	0.3665		2.728513	1
6			A.Mean		1	1.788833		1.326171	1
7			G.Mean		1	1.135999		0.880282	1

SPEC Power Benchmark

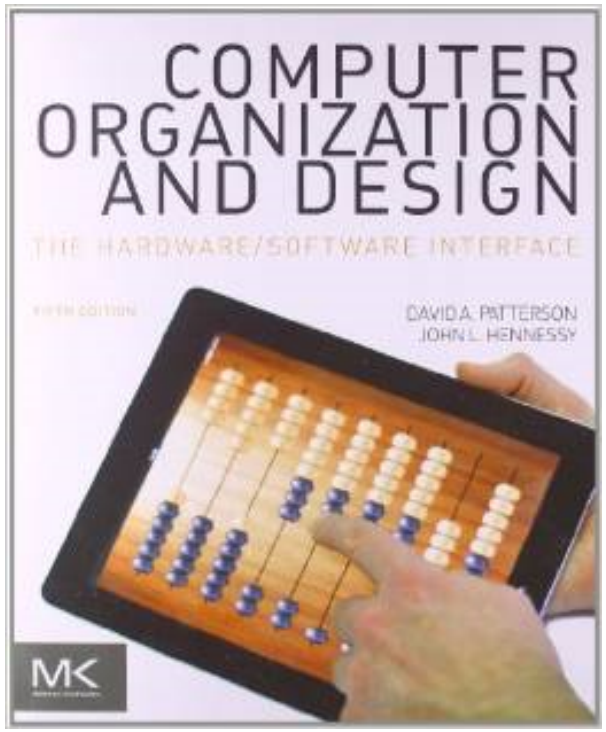
- SPECpower: benchmark to measure power
- SPEC benchmark for Java business applications (SPECJBB2005)
- Performance is measured in throughput, unit: business operations per second
- summarize in "overall ssj ops per watt"
- ssj ops $\equiv$ server side java operations

Target Load %	Performance (ssj_ops)	Average Power (watts)
100%	865,618	258
90%	786,688	242
80%	698,051	224
70%	607,826	204
60%	521,391	185
50%	436,757	170
40%	345,919	157
30%	262,071	146
20%	176,061	135
10%	86,784	121
0%	0	80
Overall Sum	4,787,166	1922
∑ssj_ops / ∑power =		2490

FIGURE 1.19 SPECpower_ssj2008 running on a dual socket 2.66 GHz Intel Xeon X5650 with 16 GB of DRAM and one 100 GB SSD disk.

Closing note

- อ่าน P&H5e, 1.6-1.9.



"I got the idea for the mouse while attending a talk at a computer conference.
The speaker was so boring that I started daydreaming and hit upon the idea"

- Doug Engelbart